

SMART EQUIPMENT MODULE (SEM)

API DETAILED REPORT

Smart Enovations (India) Private Limited.
99,100, D2, 3rd Floor, MSM Plaza
Dodd Banaswadi, Bangalore.
560043

● REST API'S:

SEM Model contains total 16 Apis with respect to all 4 controllers, which are responsible for all CRUD operations.

1. XR75 - 4 Apis
2. CoreLink - 4 Apis
3. RTN-400 - 4 Apis
4. IR33 - 4 Apis

And all 4 apis are based on periodicity and there features.

● XR75 Controller:

List of API's:

```
"sec": "http://localhost:8000/api/xr75/sec/", For 10 Seconds Data  
"min": "http://localhost:8000/api/xr75/min/", For 5 Minutes Data  
"hour": "http://localhost:8000/api/xr75/hour/", For 4 Hours Data  
"config": "http://localhost:8000/api/xr75/config/", For Configuration table Data
```

All above API's will work for all CRUD operations.

C - Create (Post Method)

R - Retrieve (Get Method)

U - Update (Put / Patch Method)

D - Destroy (Delete Method)

1. Post Method:

To test the POST method , run the web-server , go to web browser and search for <http://localhost:8000/docs/> and click on interactive, enter required data and click on request.

API - <http://localhost:8000/api/xr75/sec/>

xr75 > sec > create Interact

POST /api/xr75/sec/

Request Body

The request body should be a "application/json" encoded object, containing the following items.

Parameter	Description
probe1_value required	
probe2_value required	
probe3_value required	
probe4_value required	
real_set_point required	
probe1 required	
probe2 required	
probe3 required	
probe4 required	
probe_n required	
d11	
d12	
on_off_status	

Output: for Post Method

➡ xr75 > sec > create

POST /api/xr75/sec/ 201

Probe1 value *
1

Probe2 value *
1

Probe3 value *
1

Probe4 value *
1

Real set point *
1

Probe1 *
1

Probe2 *
1

Probe3 *
1

Probe4 *

```
{
  "id": 1,
  "probe1_value": 1,
  "probe2_value": 1,
  "probe3_value": 1,
  "probe4_value": 1,
  "real_set_point": 1,
  "probe1": 1,
  "probe2": 1,
  "probe3": 1,
  "probe4": 1,
  "probe_r": 1,
  "dl1": false,
  "dl2": false,
  "on_off_status": true,
  "def_status": true,
  "def2_status": false,
  "alarm_status": false,
  "light_status": false,
  "fan_status": true,
  "aux_status": false,
  "neutral_zone_status": true,
  "comp_status": false,
  "buzzer_status": false,
  "energy_save_status": false,
  "probe1_failure_status": false,
  "probe2_failure_status": false,
}
```

2. Get Method:

To test the Get method , run the web-server , go to web browser and search for <http://localhost:8000/docs/> and click on interactive, enter required data and click on request.

API - <http://localhost:8000/api/xr75/sec/>

xr75 > sec > list

GET /api/xr75/sec/ Interact

Query Parameters

The following parameters should be included as part of a URL query string.

Parameter	Description
limit	Number of results to return per page.
offset	The initial index from which to return the results.
ordering	Which field to use when ordering the results.

Output: For Get Method

➡ xr75 > sec > list

GET /api/xr75/sec/?limit=1 200

Limit
1
Number of results to return per page.

Offset

The initial index from which to return the results.

Ordering

Which field to use when ordering the results.

```
{
  "count": 1,
  "next": null,
  "previous": null,
  "results": [
    {
      "id": 1,
      "probe1_value": 1,
      "probe2_value": 1,
      "probe3_value": 1,
      "probe4_value": 1,
      "real_set_point": 1,
      "probe1": 1,
      "probe2": 1,
      "probe3": 1,
      "probe4": 1,
      "probe_r": 1,
      "dl1": false,
      "dl2": false,
      "on_off_status": true,
      "def_status": true,
      "def2_status": false,
      "alarm_status": false,
      "light_status": false,
      "fan_status": true,
      "aux_status": false,
      "neutral_zone_status": true,
      "comp_status": false,
    }
  ]
}
```

3. PUT / PATCH METHOD:

To test the PUT / PATCH method , run the web-server , go to web browser and search for <http://localhost:8000/docs/> and click on interact, enter required data and click on request.

API - `http://localhost:8000/api/xr75/sec/{ID}/`

Here we need to pass ID Number in the API.

PUT: PASS ALL Fields Data

PATCH : PASS Only Modified Field Data

xr75 > sec > update

Interact

PUT `/api/xr75/sec/{id}/`

Path Parameters

The following parameters should be included in the URL path.

Parameter	Description
<code>id</code> required	A unique integer value identifying this xr sec.

Query Parameters

The following parameters should be included as part of a URL query string.

Parameter	Description
<code>ordering</code>	Which field to use when ordering the results.

Request Body

The request body should be a `"application/json"` encoded object, containing the following items.

Parameter	Description
<code>probe1_value</code> required	
<code>probe2_value</code> required	
<code>probe3_value</code> required	
<code>probe4_value</code> required	

xr75 > sec > partial_update

Interact

PATCH `/api/xr75/sec/{id}/`

Path Parameters

The following parameters should be included in the URL path.

Parameter	Description
<code>id</code> required	A unique integer value identifying this xr sec.

Query Parameters

The following parameters should be included as part of a URL query string.

Parameter	Description
<code>ordering</code>	Which field to use when ordering the results.

Request Body

The request body should be a `"application/json"` encoded object, containing the following items.

Parameter	Description
<code>probe1_value</code>	
<code>probe2_value</code>	
<code>probe3_value</code>	

Output: For PUT/PATCH Method

xr75 > sec > update

Data Raw

ID *

1

A unique integer value identifying this xr sec.

Probe1 value *

2

Probe2 value *

2

Probe3 value *

2

Probe4 value *

2

Real set point *

2

Probe1 *

2

Probe2 *

-

PUT `/api/xr75/sec/1/`

200

```
{
  "id": 1,
  "probe1_value": 2,
  "probe2_value": 2,
  "probe3_value": 2,
  "probe4_value": 2,
  "real_set_point": 2,
  "probe1": 2,
  "probe2": 2,
  "probe3": 2,
  "probe4": 2,
  "probe_r": 2,
  "d11": true,
  "d12": false,
  "on_off_status": true,
  "def_status": true,
  "def2_status": true,
  "alarm_status": false,
  "light_status": true,
  "fan_status": true,
  "aux_status": false,
  "neutral_zone_status": true,
  "comp_status": true,
  "comp_status2": false,
  "buzzer_status": true,
  "energy_save_status": false,
  "probe1_failure_status": true,
  "probe2_failure_status": false
}
```

xr75 > sec > partial_update

Data Raw

ID *

1

A unique integer value identifying this xr sec.

Probe1 value

33

Probe2 value

3

Probe3 value

3

Probe4 value

3

Real set point

Probe1

Probe2

PATCH `/api/xr75/sec/1/`

200

```
{
  "id": 1,
  "probe1_value": 33,
  "probe2_value": 3,
  "probe3_value": 3,
  "probe4_value": 3,
  "real_set_point": 2,
  "probe1": 2,
  "probe2": 2,
  "probe3": 2,
  "probe4": 2,
  "probe_r": 2,
  "d11": false,
  "d12": false,
  "on_off_status": false,
  "def_status": false,
  "def2_status": false,
  "alarm_status": false,
  "light_status": false,
  "fan_status": false,
  "aux_status": false,
  "neutral_zone_status": false,
  "comp_status": false,
  "comp_status2": false,
  "buzzer_status": false,
  "energy_save_status": false,
  "probe1_failure_status": false,
  "probe2_failure_status": false
}
```

Delete Method:

To test the DELETE method , run the web-server , go to web browser and search for <http://localhost:8000/docs/> and click on interact, enter required data and click on request.

API - `http://localhost:8000/api/xr75/sec/ID/`

Here we need to pass ID Number in the API.

xr75 > sec > delete Interact

DELETE `/api/xr75/sec/{id}/`

Path Parameters

The following parameters should be included in the URL path.

Parameter	Description
<code>id</code> required	A unique integer value identifying this xr sec.

Query Parameters

The following parameters should be included as part of a URL query string.

Parameter	Description
<code>ordering</code>	Which field to use when ordering the results.

Output: For Delete Method

➡ xr75 > sec > delete Data Raw

ID *

A unique integer value identifying this xr sec.

Ordering

Which field to use when ordering the results.

DELETE `/api/xr75/sec/2/` 404

`undefined`

Close Send Request

- ◆ After Deletion ID will become undefined

Follow the Same Procedure for all other API's of XR75 and All other Controllers.

API's of other 3 Controllers:

2. CoreLink:

```
"sec": "http://localhost:8000/api/core-link/sec/", For 10 Seconds Data  
"min": "http://localhost:8000/api/core-link/min/", For 5 Minutes Data  
"hour": "http://localhost:8000/api/core-link/hour/", For 4 Hours Data  
"config": "http://localhost:8000/api/core-link/config/", For Configuration table Data
```

3. RTN400:

```
"sec": "http://localhost:8000/api/rtn400/sec/", For 10 Seconds Data  
"min": "http://localhost:8000/api/rtn400/min/", For 5 Minutes Data  
"hour": "http://localhost:8000/api/rtn400/hour/", For 4 Hours Data  
"config": "http://localhost:8000/api/rtn400/config/", For Configuration table Data
```

4. IR33:

```
"sec": "http://localhost:8000/api/ir33/sec/", For 10 Seconds Data  
"min": "http://localhost:8000/api/ir33/min/", For 5 Minutes Data  
"hour": "http://localhost:8000/api/ir33/hour/", For 4 Hours Data  
"config": "http://localhost:8000/api/ir33/config/", For Configuration table Data
```

And all above API's are connect to Third party application (Front-end) through a library call AXIOS.

Thank You